



Universidad de Oriente
Núcleo de Monagas
Escuela de Ingeniería y Ciencias Aplicadas
Departamento de Ingeniería de Sistemas
Estructura de Datos (0723123)

ALFA - APLICACIÓN DE INSCRIPCIÓN O REGISTRO DE USUARIOS

Profesor (a):

MSc. Anibal Fariñas

Bachilleres:

CI. 28.474.501 Magda Salazar

CI. 30.955.531 Jahilibeth González

CI. 31.407.610 Nahomi Rivas

CI. 28.512.009 Rubén Molina

CI. 30.386.037 Oma lanz

Maturín, julio de 2026

DOCUMENTACIÓN DEL SISTEMA DE INSCRIPCIÓN DE USUARIOS

Comentarios y documentación del código fuente

El código está organizado en varias clases que cumplen funciones específicas. A continuación, se describen las principales:

Clase `Aplicacion\_de\_inscripcion\_de\_usuario` (Bootstrap)

```
``java
/**
 * Punto de entrada principal (Bootstrap) para la aplicación.
 * Inicializa el Gestor de Usuarios único y monta la interfaz gráfica avanzada.
 */
public class Aplicacion_de_inscripcion_de_usuario {

    // Instancia singleton del gestor de usuarios, se crea al cargar la clase
    private static final GestorUsuarios GESTOR_UNICO = new GestorUsuarios();

    /**
     * Método principal que inicia la aplicación en el Event Dispatch Thread de Swing.
     */
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            // Inicializa la ventana principal con el gestor compartido
            VentanaGrafica ventana = new VentanaGrafica(GESTOR_UNICO);
            ventana.setVisible(true);
            System.out.println("¡Sistema de Inscripción iniciado y sincronizado con éxito!");
        });
    }
}
```

```
}  
...  

```

Qué hace:

Inicia la aplicación creando la ventana principal y asegurando que la carga de interfaz gráfica se realice en el hilo adecuado (EDT). Además, mantiene un gestor de usuarios singleton.

Limitaciones:

No maneja errores en la carga de recursos gráficos, asume que `GestorUsuarios` funciona correctamente.

Clase `GestorUsuarios`

```
```java  
/**
 * Clase que gestiona toda la lógica de almacenamiento, búsqueda y mantenimiento
de los usuarios.
 * Incluye estructuras de datos en memoria, persistencia en archivo y operaciones
CRUD.
 */
public class GestorUsuarios implements Serializable {
 // Variables internas: hash, lista enlazada, pilas, colas, árboles
 private HashMapUsuarios hashUsuarios;
 private ListaEnlazadaUsuarios listaUsuarios;
 private Stack<String> pilaAcciones; // Para deshacer acciones
 private LinkedList<Usuario> colaEspera; // Cola de usuarios en espera
 private ArbolBinario arbolBinario; // Árbol binario ordenado por nombre
 private ArbolAVL arbolAVL; // Árbol AVL ordenado por ID

 /**
 * Constructor: inicializa estructuras y carga datos persistentes

```

```

 */
 public GestorUsuarios() {
 inicializarEstructurasVacias();
 cargarDatos();
 }

 /**
 * Guarda en disco el estado actual de los usuarios
 */
 public void guardarDatos() {
 try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(FILE_PATH))) {
 oos.writeObject(this.listaUsuarios);
 } catch (IOException e) {
 System.out.println("ERROR al guardar datos: " + e.getMessage());
 }
 }

 /**
 * Carga datos desde el archivo binario, reconstruye estructuras en memoria
 */
 public void cargarDatos() {
 File archivo = new File(FILE_PATH);
 if (!archivo.exists()) return; // No hay datos guardados

 try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(archivo))) {
 this.listaUsuarios = (ListaEnlazadaUsuarios) ois.readObject();
 reconstruirEstructurasAuxiliares();
 } catch (Exception e) {
 System.out.println("ERROR al cargar datos, reiniciando: " + e.getMessage());

```

```

 inicializarEstructurasVacias();
 }
}

/**
 * Reconstruye estructuras auxiliares después de cargar
 */
private void reconstruirEstructurasAuxiliares() {
 hashUsuarios = new HashMapUsuarios();
 pilaAcciones = new Stack<>();
 colaEspera = new LinkedList<>();
 arbolBinario = new ArbolBinario();
 arbolAVL = new ArbolAVL();

 if (this.listaUsuarios == null) return;

 for (Usuario u : this.listaUsuarios.obtenerTodosLosUsuarios()) {
 if (u != null) {
 hashUsuarios.insertar(u);
 colaEspera.add(u);
 arbolBinario.insertar(u);
 arbolAVL.insertar(u);
 }
 }
}

/**
 * Valida que los datos de un usuario sean correctos y únicos
 */
public boolean validarUsuario(String id, String nombre, String email) {
 if (id == null || nombre == null || email == null) return false;

```

```

 if (id.isEmpty() || nombre.isEmpty() || email.isEmpty()) return false;
 if (hashUsuarios.buscar(id) != null) return false; // ID duplicado
 if (!email.contains("@")) return false; // Validación simple email
 return true;
 }
 // Otros métodos de CRUD y lógica de negocio...
}
```

```

Qué hace:

Gestiona datos en memoria, persistencia y estructura auxiliar. Permite agregar, eliminar, buscar y deshacer cambios.

Limitaciones:

No maneja errores complejos, asume que `Usuario` y otras estructuras funcionan correctamente.

Clase `ArbolAVL`

```

```java
/**
 * Nodo del Árbol AVL para almacenaje de usuarios con balanceo
 */
public class NodoAVL implements Serializable {
 public Usuario usuario;
 public NodoAVL izquierdo, derecho;
 public int altura;

 public NodoAVL(Usuario usuario) {
 this.usuario = usuario;
 this.izquierdo = null;
 this.derecho = null;
 }
}
```

```

```

        this.altura = 1; // Nodo hoja
    }
}
```

```

### **Qué hace:**

Representa los nodos del árbol AVL, facilitando el balanceo y búsqueda eficiente por ID.

### **Limitaciones:**

No se muestra la implementación completa de rotaciones y balanceo en este fragmento.

### **Clase `VentanaGrafica`**

```

```java
/**
 * Ventana principal con interfaz gráfica Swing para gestionar usuarios.
 * Incluye botones para registrar, eliminar y administrar usuarios.
 */
public class VentanaGrafica extends JFrame {
    // Componentes y gestor
    private JPanel contentPane;
    private JTextField txtId, txtNombre, txtEmail;
    private GestorUsuarios gestor;

    /**
     * Constructor: configura la ventana, componentes y eventos
     */
    public VentanaGrafica(GestorUsuarios gestorCompartido) {
        this.gestor = gestorCompartido;
        setSize(850, 500);
    }
}
```

```

```

 setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
 setLocationRelativeTo(null);
 // Configuración de interfaz, carga imagen, botones, etc.
 // ...
 }
 // Métodos para mostrar formularios, validar, limpiar
}
'''

```

### **Qué hace:**

Proporciona interfaz para registrar, eliminar usuarios y administrar la vista.

## **2. Casos manejados y limitaciones**

### **Casos manejados:**

- Registro de usuarios con validación básica.
- Eliminación de usuarios por ID.
- Visualización de todos los usuarios.
- Persistencia en archivos binarios.
- Deshacer la última acción de registro/eliminación.
- Acceso administrativo mediante clave fija.
- Interfaz gráfica para interacción amigable.

### **Casos NO manejados (limitaciones):**

- Validación avanzada de emails.
- Manejo de errores en archivos (se reintentan o informa al usuario).
- Seguridad avanzada (clave de administrador fija).
- Deshacer eliminación de usuarios (solo registro).
- Multihilo para múltiples usuarios simultáneos.
- Funciones de actualización de datos existentes (solo agregar y eliminar).
- Persistencia en bases de datos externas (solo archivos binarios).